

Assistant AI - AI-powered FAQ and customer support automation

Description

The **AI FAQ Assistant** is a robust, secure, and intelligent REST API platform designed to simplify FAQ creation and automate customer support using artificial intelligence.

The application is developed using **Node.js and Express.js**, with **MongoDB** as the primary database and **Mongoose ODM** for data modelling and schema validation.

The system provides **JWT-based authentication and authorization** to secure user sessions and protected API routes. It also uses **crypt** to securely hash user passwords before storing them in the database.

The backend supports important features such as **AI-powered FAQ generation, semantic FAQ search, dynamic context parsing, and centralized error handling**. Google Gemini AI is integrated to generate questions, answers, and topic suggestions automatically.

Overall, the AI FAQ Assistant helps reduce manual customer-support work by allowing users to create, manage, search, and generate FAQs efficiently through a secure REST API.

Scenario-Based Case Study

Background

A customer support team receives a large number of repetitive questions from users every day. Managing these questions manually, preparing accurate answers, organizing FAQs into categories, and keeping the information updated can consume significant time and effort. The support team needs a centralized system that can simplify FAQ management and provide quick access to relevant information.

Problem

The existing manual approach creates several challenges:

- Difficulty in creating and maintaining a large number of FAQs.
- Time-consuming process of preparing answers for repetitive customer questions.
- Difficulty in finding relevant FAQs quickly from a large collection of records.

- Lack of proper categorization for organizing FAQ content.
- Limited automation for generating new questions and answers.
- Need for secure user authentication and controlled access to FAQ management operations.

Solution

The **AI FAQ Assistant** provides a centralized and secure REST API that addresses these challenges through automated FAQ management and AI assistance. Authenticated users can create, update, and delete FAQ records, while categories help organize the content efficiently.

The system integrates **Google Gemini AI** to generate relevant FAQ questions, answers, and category suggestions based on a given topic. Users can also search existing FAQs using keywords to quickly retrieve relevant information.

JWT-based authentication protects private operations, while crypt securely handles user passwords. Mongoose schema validation and centralized error handling help maintain data integrity and provide reliable API responses.

Usage

- **Admin:** Manages users, FAQs, categories, and monitors AI-generated content.
- **Content Creator:** Creates, edits, deletes, organizes, and generates FAQs using AI.
- **Authenticated User:** Views FAQs, searches for information, and generates AI-assisted answers.
- **Public User:** Views and searches published FAQs without access to protected operations.

Outcome

The AI FAQ Assistant reduces manual effort in FAQ management by combining automated content generation, organized FAQ storage, intelligent search, and secure access control. It provides a structured backend solution that improves the efficiency of managing and retrieving customer-support information.

1. Software Requirements

- **Operating System:** A Windows, macOS, or Linux environment is required to develop and run the application.
- **Node.js:** The backend application requires Node.js version 18 or later.
- **npm:** npm is used to install and manage the packages required by the project.
- **Express.js:** Express.js is used to develop the server and handle API requests.
- **MongoDB:** MongoDB is used to store user accounts, FAQ records, and other application data.

- **Mongoose:** Mongoose is used to define database schemas and interact with MongoDB.
- **Postman / Thunder Client:** These tools are used to test and verify the API functionality.
- **Visual Studio Code:** VS Code can be used as the development environment for writing and managing the project code.

2. Hardware Requirements

- **Processor:** A modern dual-core or quad-core processor is sufficient for developing and running the application.
- **Memory:** 8 GB of RAM is recommended for smooth execution of the backend, database, and development tools.
- **Storage:** At least 5 GB of free disk space is recommended for the project files, dependencies, database tools, and development environment.
- **Internet Connection:** A stable internet connection is required for accessing cloud-based AI services and other online resources used by the application.

processing topic-based requests before the generated content is returned or stored.

Request Flow

The overall request flow begins when an API client sends a request to the Express server. The request is routed through authentication when required, then passed to the appropriate application service. Database-related operations are handled through Mongoose and MongoDB, while AI-related requests are forwarded to the Google Gemini service. After processing, the result is returned to the API client as a structured response.

Architecture Flow

API Client → Express Server → Authentication / Application Services → Database Layer

Application Services → Google Gemini AI → Generated FAQ Content

This layered structure keeps the system modular and makes the backend easier to maintain, extend, secure, and test.

Entity Relationship Diagram Description

The Entity Relationship Diagram represents the main data entities used by the AI FAQ Assistant and illustrates how these entities are connected within the database. It provides a clear view of how user information, FAQ records, categories, and AI-generated content are organized and related.

1. User Entity

The User entity maintains the details of registered users who interact with the system.

Main attributes:

1. FAQ Entity

The FAQ entity stores the questions and answers managed by the application.

Main attributes:

- FAQ ID
- Question
- Answer
- Category
- Created By
- Created Date
- Updated Date

Each FAQ is associated with a user who creates it and is organized under a category.

2. Category Entity

The Category entity is used to group FAQs according to their topics or subject areas.

Main attributes:

- Category Name
- Description
- Total FAQs

One category can contain multiple FAQ records.

3. AI Generation Entity

AI Generation represents the logical data produced when the system uses Google Gemini to generate FAQ content. This information can later be stored as an FAQ record.

Main attributes:

- Topic
- Generated Question
- Generated Answer
- Generated Category
- Generated Date

A user can generate multiple AI-assisted FAQ contents.

Relationship Description

User → FAQ (One-to-Many):

A single user can create and manage multiple FAQ records, while each FAQ is associated with its creator.

Category → FAQ (One-to-Many):

A category can contain several FAQ records, allowing related questions and answers to be organized together.

User → AI Generated FAQ (One-to-Many):

A user may generate multiple FAQ contents using the AI service. These generated results can later be saved as FAQ records.

FAQ → Search Results (Logical Relationship):

FAQ records can be retrieved through keyword or semantic search based on the user's query.

Overall, the ER diagram provides a structured representation of the data relationships that support authentication, FAQ management, category organization, AI-assisted generation, and information retrieval.

Key Structural Mappings

The database entities of the AI FAQ Assistant are connected through logical relationships that support user management, FAQ organization, and AI-assisted content generation.

User – FAQ Relationship

A registered user can be associated with multiple FAQ records. Each FAQ maintains a reference to the user responsible for creating or managing it.

Category – FAQ Relationship

A single category can group several FAQ entries. This relationship helps organize questions and answers according to their respective topics.

User – AI Generated FAQ Relationship

One user may generate multiple FAQ contents with the assistance of the AI service. The generated information can subsequently be stored as FAQ data.

FAQ – Search Relationship

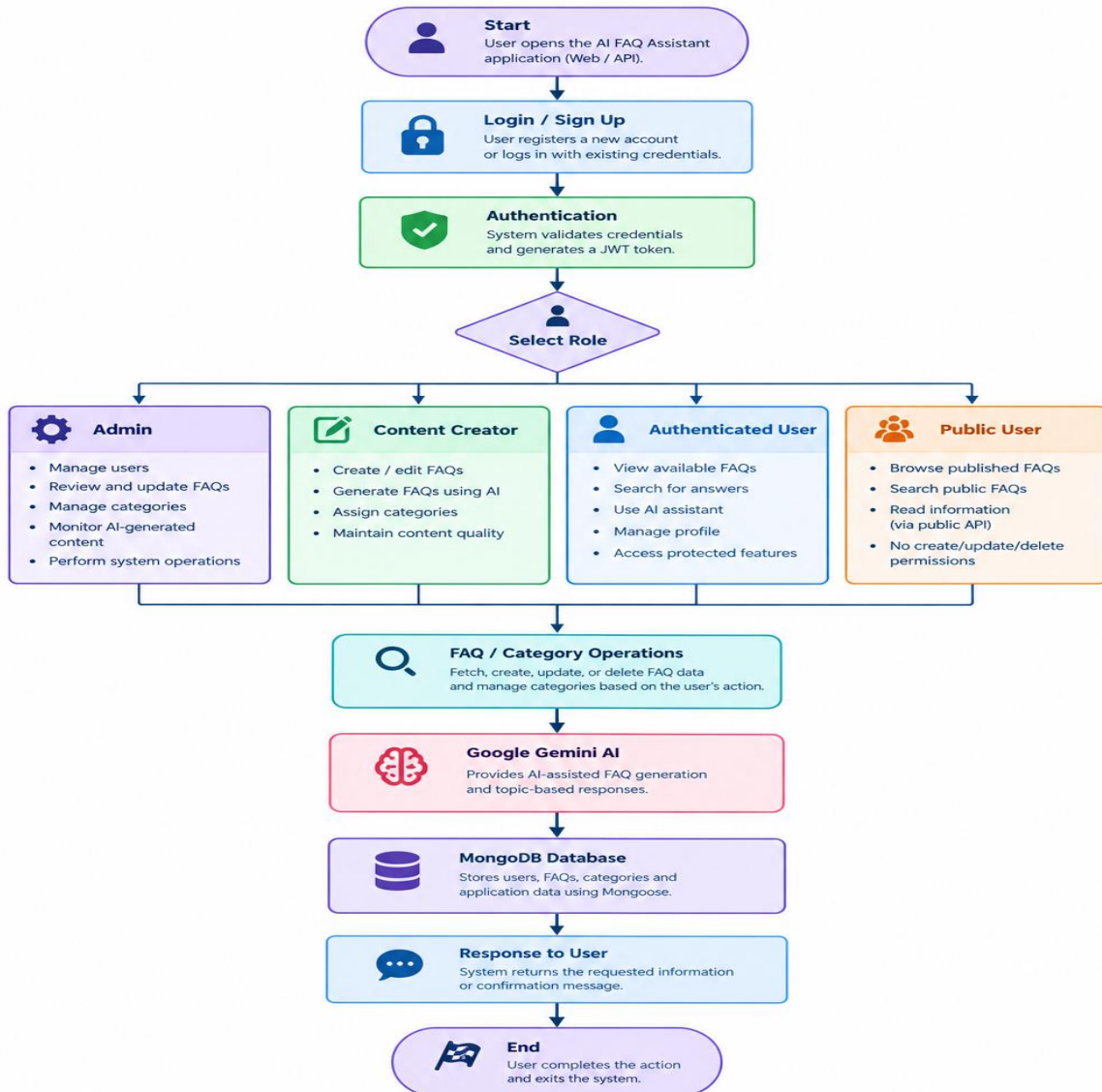
FAQ records are accessed through search operations. User queries can retrieve relevant FAQ information using keyword-based or semantic matching.

These mappings provide the logical connections required for maintaining organized FAQ data, associating content with users and categories, and supporting AI-assisted FAQ generation.

User Flow Diagram

AI FAQ Assistant

This diagram illustrates the flow of a user interacting with the AI FAQ Assistant, from accessing the system to getting the required information or performing specific actions based on their role.



Architecture Pattern

The AI FAQ Assistant follows the **Model-View-Controller (MVC) architecture pattern** to organize the backend into separate and manageable components. This structure improves code organization, maintainability, and separation of responsibilities.

1. Model

The Model layer manages the application's data and database structure. It uses **Mongoose** to define schemas and communicate with **MongoDB**.

Responsibilities:

- Define User, FAQ, and Category data structures
- Perform database operations

- Validate stored data
- Retrieve and update application records

2. Controller

The Controller layer handles incoming API requests and connects them with the appropriate application logic.

Responsibilities:

- Receive API requests
- Validate request information
- Call required services or database operations
- Process the results
- Send appropriate API responses

3. View

Since the project is a REST API rather than a traditional web application, the View layer is represented by the **API response and routing layer**.

Responsibilities:

- Define API endpoints
- Return JSON responses
- Provide success and error responses
- Communicate results to API clients such as Postman or other applications

4. Supporting Components

The MVC structure is supported by additional backend components:

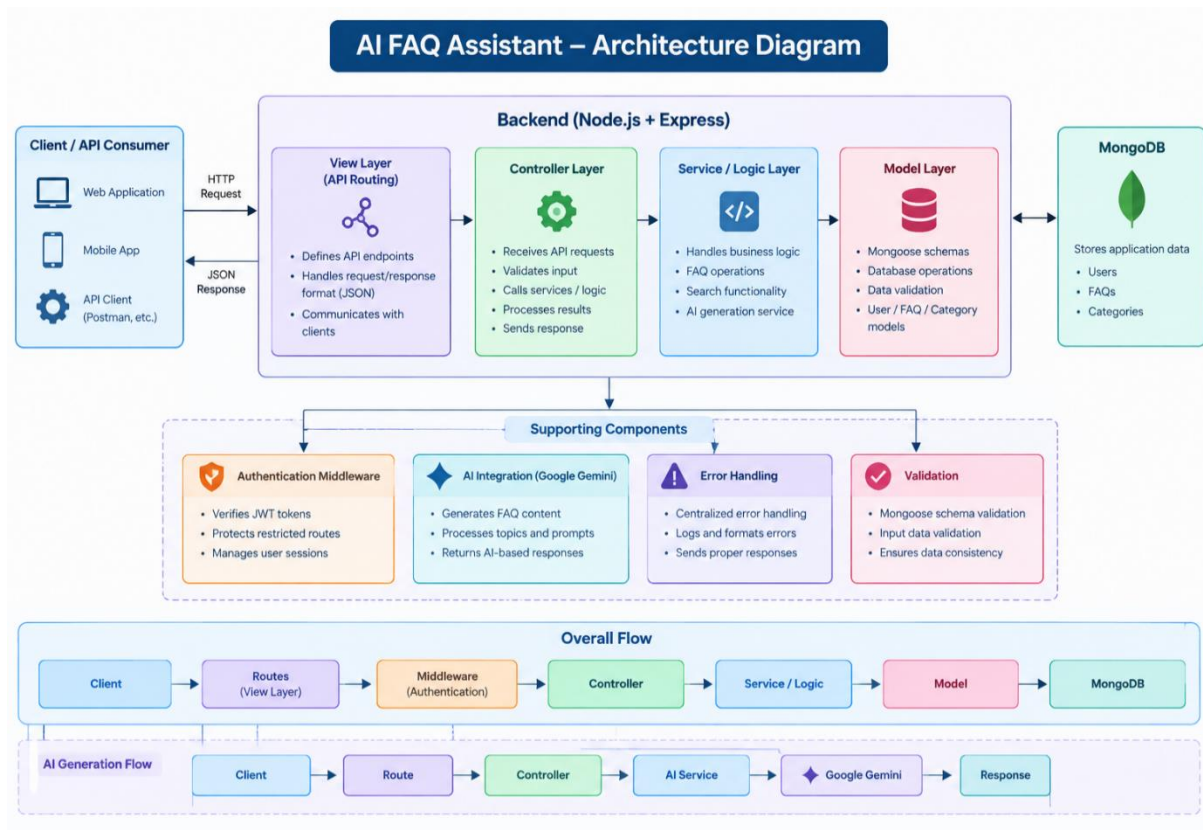
- **Authentication Middleware** – verifies JWT tokens and protects restricted routes.
- **Service/Logic Layer** – handles FAQ operations, search functionality, and AI-based FAQ generation.
- **AI Integration** – connects the application with **Google Gemini** for generating FAQ content.
- **Error Handling** – provides centralized handling of application and API errors.

Architecture Flow

API Client → Routes → Middleware → Controller → Service/Logic → Model → MongoDB

For AI-based FAQ generation:

API Client → Route → Controller → AI Service → Google Gemini → Response



Epic 2: Project Setup and Configuration

This phase focuses on preparing the development environment and establishing the basic project structure for the AI FAQ Assistant. The required backend dependencies, source folders, and configuration files are organized before implementing the application features.

```

Terminal  Help

... {} package.json X ⓘ REA

src > JS app.js > ...
4   const { errorHa
5
6   // Load environ
7   dotenv.config()
8

```

Creating Project Folder

The AI FAQ Assistant project is organized within a dedicated project directory to keep the backend files and source code structured.

Steps to Create the Project Folder

1. Create a new folder with the project name **AI-FAQ-Assistant**.
2. Open the newly created project folder in **Visual Studio Code**.
3. Inside the project folder, create a subfolder named **src**. This folder is used to organize the main source code of the application.
4. Open the project folder terminal in Visual Studio Code to initialize and configure the Node.js backend project.

Epic 3: Backend Development

The core system scripts form the main backend foundation of the AI FAQ Assistant. Each script is responsible for a specific part of the application, such as starting the server, configuring Express, securing API requests, and handling errors.

1. Server Setup – src/server.js

The server.js file acts as the main entry point of the backend application. It loads the required configuration, establishes the MongoDB connection, and starts the Express server on the configured port.

Main responsibilities:

- Load environment configuration
- Connect the application with MongoDB
- Initialize the Express application
- Start the backend server

2. Express Application Setup – src/app.js

The app.js file configures the Express application and prepares it to handle API requests.

Main responsibilities:

- Initialize Express
- Configure JSON request handling

3. Authentication Middleware – src/middleware/authMiddleware.js

The authentication middleware protects restricted API endpoints. It reads the JWT token provided with the request and verifies whether the user is authorized to access the requested resource.

Main responsibilities:

- Read the authorization token
- Verify the JWT
- Identify the authenticated user
- Protect private API routes

- Reject unauthorized requests

4. Error Handling Middleware – `src/middleware/errorMiddleware.js`

The centralized error-handling middleware manages errors generated during API execution. It provides consistent responses instead of allowing unhandled errors to reach the client.

Main responsibilities:

- Capture application errors
- Handle database and API errors
- Return standardized error responses
- Prevent unnecessary internal error details from being exposed

Core Backend Flow

Client Request → Express App → Authentication Middleware → Route → Controller/Logic → Database or AI Service → Response

These core scripts provide the basic execution structure required for the AI FAQ Assistant backend.

Epic 4: Database Configuration

Database configuration

The AI FAQ Assistant uses **MongoDB** as its database and **Mongoose** as the ODM layer. The database configuration is responsible for storing user information, FAQ records, and maintaining the required data structure and validation rules.

1. MongoDB Connection

The backend application establishes a connection with MongoDB through Mongoose. The database connection details are maintained through environment configuration so that they are separated from the application source code.

2. User Schema – `src/models/User.js`

The User model defines the structure of user information stored in MongoDB.

Main fields include:

- User ID
- Name
- Email
- Password
- Role
- Created date
- Updated date

The password is protected using hashing before the user information is stored in the database.

3. FAQ Schema – src/models/FAQ.js

The FAQ model stores the questions and answers managed by the application.

Main fields include:

- FAQ ID
- Question
- Answer
- Category
- Created By
- Created date
- Updated date

The createdBy field connects an FAQ record with the user who created it.

4. Data Validation

Mongoose schema validation is used to ensure that required information is provided before records are stored in MongoDB. This helps maintain consistency and data integrity.

5. Database Relationship

The main relationship in the database is:

User → FAQ (One-to-Many)

A single registered user can create and manage multiple FAQ records.

Database Flow

Application → Mongoose → MongoDB

Through this configuration, the AI FAQ Assistant can securely store and retrieve user and FAQ information while maintaining a consistent database structure.

```
1 const errorHandler = (err, req, res, next) => {
2   let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3   let message = err.message;
4
5   // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
6   if (err.status) {
7     statusCode = err.status;
8   }
9
10  // Mongoose duplicate key error (e.g. duplicate email)
11  if (err.code === 11000) {
12    statusCode = 400;
13    const field = Object.keys(err.keyValue)[0];
14    message = `Duplicate field value entered: ${field}. Please use another value.`;
15  }
16
17  // Mongoose validation error
18  if (err.name === 'ValidationError') {
19    statusCode = 400;
20    message = Object.values(err.errors)
21      .map((val) => val.message)
22      .join(', ');
23  }
24
25  // Mongoose bad ObjectId
26  if (err.name === 'CastError') {
27    statusCode = 404;
28    message = `Resource not found with id of ${err.value}`;
29  }
30
31  // Only log stack trace to the console for unexpected 500 Server Errors
32  if (statusCode === 500) {
33    console.error(err.stack);
34  } else {
35  }
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

The FAQ schema defines the structure of FAQ records stored in the MongoDB database. It uses **Mongoose** to specify the required fields and validation rules for each FAQ document.

Main Fields

- **question** – Stores the FAQ question and is required.
- **answer** – Stores the corresponding answer and is required.
- **category** – Identifies the category associated with the FAQ.
- **createdBy** – References the User collection to identify the user who created the FAQ.
- **createdAt** – Records when the FAQ was created.
- **updatedAt** – Records when the FAQ was last modified.

Purpose

The FAQ schema ensures that FAQ records follow a consistent structure before they are stored in MongoDB. It also maintains the relationship between an FAQ and its creator through the **createdBy** reference.

Data Flow:

FAQ Request → FAQ Schema Validation → Mongoose → MongoDB

```
src > middleware > JS errorMiddleware.js > ...
1  const errorHandler = (err, req, res, next) => {
2      let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3      let message = err.message;
4
5      // Respect status code set on the error object (e.g., 429 from Gemini SDK or 408/403/404)
6      if (err.status) {
7          statusCode = err.status;
8      }
9
10     // Mongoose duplicate key error (e.g. duplicate email)
11     if (err.code === 11000) {
12         statusCode = 400;
13         const field = Object.keys(err.keyValue)[0];
14         message = `Duplicate field value entered: ${field}. Please use another value.`;
15     }
16
17     // Mongoose validation error
18     if (err.name === 'ValidationError') {
19         statusCode = 400;
20         message = Object.values(err.errors)
21             .map((val) => val.message)
22             .join(', ');
23     }
24
25     // Mongoose bad ObjectId
26     if (err.name === 'CastError') {
27         statusCode = 404;
28         message = `Resource not found with id of ${err.value}`;
29     }
30
31     // Only log stack trace to the console for unexpected 500 Server Errors
32     if (statusCode === 500) {
33         console.error(err.stack);
34     } else {
35     }
```

API Router Interface Endpoints

1. Public Authentication Operations

File: src/routes/authRoutes.js

This router manages user authentication-related operations. It provides public endpoints for user registration and login.

Main Endpoints:

- POST /api/auth/register – Creates a new user account.
- POST /api/auth/login – Authenticates an existing user and provides a JWT token.

2. Protected FAQ Operations

File: src/routes/faqRoutes.js

This router handles FAQ resource operations. Protected operations require a valid JWT Bearer token.

Main Operations:

- Create a new FAQ
- Retrieve FAQ records
- Update FAQ information
- Delete FAQ records
- Search FAQs using query parameters

Example Endpoint:

POST /api/faqs

Access: Private – Bearer JWT Token

3. AI Automation Operations

File: src/routes/aiRoutes.js

This router provides AI-powered FAQ generation functionality through the integrated Google Gemini service.

Main Endpoint:

POST /api/ai/generate-faq

Access: Private – Bearer JWT Token

The endpoint receives a topic or context and processes it through the AI service to generate FAQ content.

API Routing Flow

Client Request → Express Router → Authentication Middleware → Controller → Service/Model → Response

This routing structure keeps authentication, FAQ management, and AI operations modular and easier to maintain.

3. AI Automation Operations (src/routes/aiRoutes.js)

File: src/routes/aiRoutes.js

The AI Automation Router manages AI-powered FAQ generation within the application. It provides a protected API endpoint that allows authenticated users to submit a topic or context and receive automatically generated FAQ content.

Main Endpoint

Route: POST /api/ai/generate-faq

Access: Private – Bearer JWT Token

The request requires a valid authentication token. The submitted topic is forwarded to the AI processing service, which uses the integrated **Google Gemini SDK** to generate relevant FAQ content.

Request Data

The endpoint accepts a topic or context in the request body.

Example:

```
{
```

```
"topic": "Mongoose schema indexing validation runtime workflow optimization"
```

```
}
```

Processing Flow

Client Request → AI Router → Authentication Check → AI Service → Google Gemini → Generated FAQ Response

The aiRoutes.js router separates AI-related operations from authentication and FAQ resource routes, making the backend structure modular and easier to maintain.

Epic 5: Project Executions

Running the API Application

The completed AI FAQ Assistant backend can be executed in a development environment to verify the functionality of the REST API.

Step 1: Install API Testing Tool

Install **Postman** or the **Thunder Client** extension in Visual Studio Code for testing the API endpoints.

Step 2: Start the Backend Server

Open the project folder terminal and execute:

```
npm run dev
```

The command starts the backend development server and makes the API routes available for testing.

API Testing Specification

The API functionality can be verified by sending requests to the configured endpoints.

1. User Registration

Route: POST /api/auth/register

Access: Public

This endpoint creates a new user account.

Request Body:

```
{
```

```
  "name": "Priya Sharma",
```

```
  "email": "priya@writeflow.com",
```

```
  "password": "securepassword123"
```

```
}
```

2. User Login

Route: POST /api/auth/login

Access: Public

This endpoint verifies the user's login credentials and generates an authentication token.

Request Body:

```
{  
  "email": "priya@writeflow.com",  
  "password": "securepassword123"  
}
```

3. Create FAQ

Route: POST /api/faqs

Access: Private – Bearer JWT Token

This endpoint allows an authenticated user to create a new FAQ record.

Request Body:

```
{  
  "question": "How do I configure custom category tags?",  
  "answer": "Navigate to the settings panel and configure the required category tags.",  
  "category": "Configuration"  
}
```

4. Search FAQ

Route: GET /api/faqs/search?q=configure

Access: Public

This endpoint searches the available FAQ records using the supplied query keyword and returns matching results.

5. AI FAQ Generation

Route: POST /api/ai/generate-faq

Access: Private – Bearer JWT Token

This endpoint generates FAQ content using the integrated AI service.

Request Body:

```
{
```

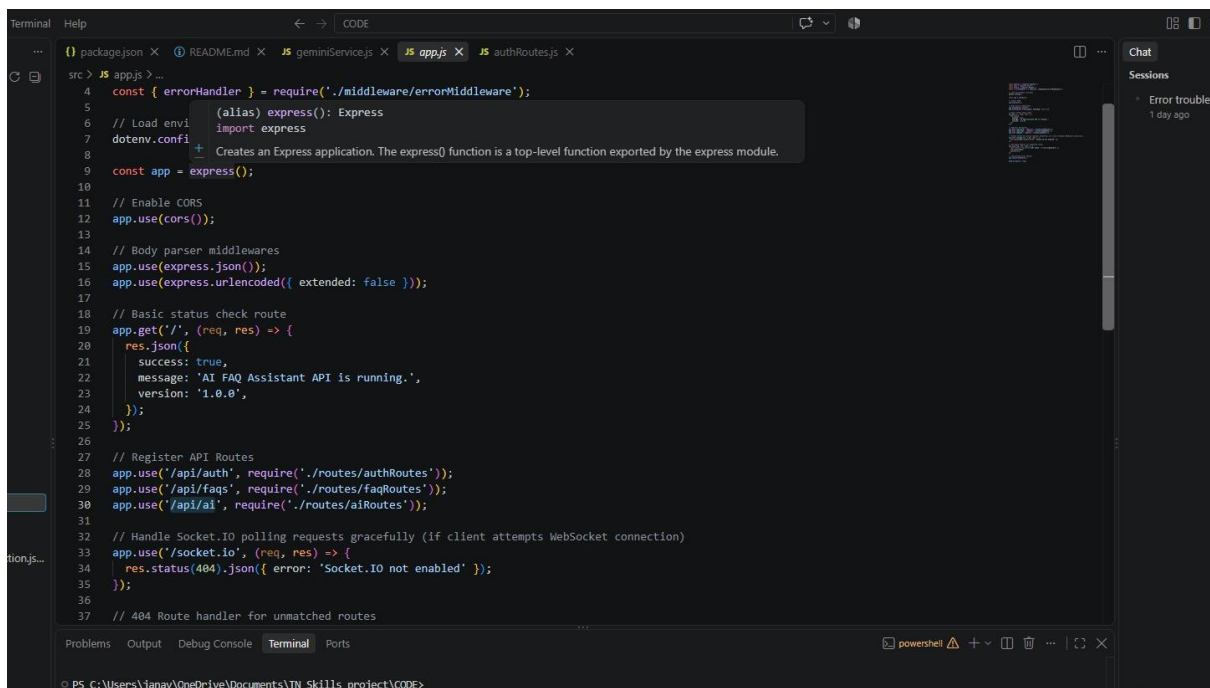
"topic": "Mongoose schema indexing validation runtime workflow optimization"

}

Execution Flow

Start Server → Authenticate User → Create/Access FAQ → Search FAQ → Generate AI FAQ → Return JSON Response

The execution phase confirms that the authentication, FAQ management, search functionality, database operations, and AI-powered generation routes are working together as a complete backend system.



```
1 const { errorHandler } = require('./middleware/errorMiddleware');
2
3 (alias) express(): Express
4 // Load environment variables from .env file
5 import express
6 // Creates an Express application. The express() function is a top-level function exported by the express module.
7
8 const app = express();
9
10 // Enable CORS
11 app.use(cors());
12
13 // Body parser middleware
14 app.use(express.json());
15 app.use(express.urlencoded({ extended: false }));
16
17 // Basic status check route
18 app.get('/', (req, res) => {
19   res.json({
20     success: true,
21     message: 'AI FAQ Assistant API is running.',
22     version: '1.0.0',
23   });
24 });
25
26 // Register API Routes
27 app.use('/api/auth', require('./routes/authRoutes'));
28 app.use('/api/faqs', require('./routes/faqRoutes'));
29 app.use('/api/ai', require('./routes/aiRoutes'));
30
31 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
32 app.use('/socket.io', (req, res) => {
33   res.status(404).json({ error: 'Socket.IO not enabled' });
34 });
35
36 // 404 Route handler for unmatched routes
37
```

3. Resource Entity Creation (FAQ)

Route: POST /api/faqs

Access: Private – Bearer JWT Token

This endpoint is used to create a new FAQ resource in the system. Only authenticated users with a valid JWT token can access this protected operation.

The request contains the FAQ question, answer, and category information. After validation, the FAQ record is stored in the MongoDB database through the Mongoose data layer.

Request Body:

{

"question": "How do I configure custom category tags?",

"answer": "Navigate to the settings panel, append keyword tags arrays, and save the schema.",

```
"category": "Configuration"
```

```
}
```

Execution Flow:

Client Request → JWT Authentication → FAQ Validation → Mongoose → MongoDB → JSON Response

This operation demonstrates secure FAQ creation while maintaining structured data validation and database persistence.

4. Keyword Resource Entity Fetching (Search)

4. Keyword Resource Entity Fetching (Search)

Route: GET /api/faqs/search?q=configure

Access: Public

This endpoint is used to search FAQ resources using a keyword provided through the query parameter. Users can retrieve relevant FAQ records without requiring authentication.

The search request sends a keyword such as configure, and the backend processes the query to identify matching FAQ content.

Output :

The screenshot displays a web browser window with a REST client interface. The URL bar shows `http://localhost:5000/api/auth/profile`. The 'Auth' tab is selected, showing a Bearer Token. The 'Response' tab shows a successful status (200 OK) with a JSON response. The terminal at the bottom shows the command `npm start` being executed, and the server output indicates that the application is running on port 5000.

```
GET http://localhost:5000/api/auth/profile Send
Status: 200 OK Size: 243 Bytes Time: 35 ms
Response Headers Cookies Results Docs
1 {
2   "success": true,
3   "message": "Profile retrieved successfully",
4   "data": {
5     "_id": "6aad6073e1ad295138778812",
6     "name": "janavikaa",
7     "email": "24cs035janavikaa@gmail.com",
8     "createdAt": "2026-09-18T16:01:55.878Z",
9     "updatedAt": "2026-09-18T16:01:55.878Z",
10    "__v": 0
11  }
12 }
```

```
PS C:\Users\janav\OneDrive\Documents\TN Skills project\CODE> npm start
> ai-faq-assistant-api@1.0.0 start
> node src/server.js

MongoDB Connected: 127.0.0.1
Server running in development mode on port 5000
```

Example Request:

GET /api/faqs/search?q=configuconfiguration

Execution Flow:

Client Request → Search Query Parameter → FAQ Search Logic → Matching FAQ Records → JSON Response

This operation supports faster FAQ information retrieval by allowing users to search the available FAQ resources using relevant key